Program Studi Sains Data
Fakultas Sains
Institut Teknologi Sumatera

Machine Learning Operations (MLOps)

Pertemuan 6

Hands-on 2: Model Experimentation & Tracking

Nama : Nadia Faraj Alyafaatin Simbolon
NIM : 123450092
Kelas : RC

1. Skenario: Deteksi Tumor Ganas (Breast Cancer Wisconsin)

Eksperimen ini membangun model klasifikasi yang membantu dokter memprioritaskan pemeriksaan lanjutan untuk tumor yang kemungkinan ganas (malignant), dengan seluruh eksperimen dilacak memakai MLflow.

Aspek	Keputusan (ditetapkan sebelum eksperimen)
Dataset	sklearn.datasets.load_breast_cancer, 569 sampel, 30 fitur numerik
Kelas positif	1 = malignant (label asli dibalik: 1 - target)
Optimizing metric	Recall, karena false negative (tumor ganas terlewat) sangat mahal
Satisficing metric	Precision ≥ 0,90, agar dokter tidak dibanjiri alarm palsu
Protokol evaluasi	Train 80% (5-fold Stratified CV untuk seleksi), test 20% hanya sekali di akhir
Algoritma	Dummy, Logistic Regression, KNN, Decision Tree, Random Forest, SVC, Gradient Boosting

Hipotesis

- **H1:** Model linear sederhana sudah jauh lebih baik daripada baseline dummy.
- **H2:** Standardisasi fitur meningkatkan recall model berbasis jarak/margin (LR, KNN, SVC).
- **H3:** Tuning hyperparameter SVC meningkatkan recall tanpa melanggar precision ≥ 0,90.

Alur eksperimen

1. **Setup:** struktur proyek dan tracking MLflow.
2. **L1:** split tetap dan versi data (hash MD5).
3. **Fungsi logging standar:** run_experiment() dipakai semua eksperimen.

4. **L2:** baseline (H1, H2).
5. **L3:** perbandingan algoritma sederhana.
6. **L4:** tuning SVC dengan nested runs (H3).
7. **L5:** pemilihan kandidat terbaik secara programatik.
8. **L6:** evaluasi final di test set (sekali saja).
9. **L7:** Model Registry dan quality gate.
10. **L8:** serving model dari registry.
11. **L9:** reproduce, dari model produksi kembali ke asal-usulnya.

2. Struktur Proyek dan Setup

Proyek ini dijalankan di Google Colab. Library dipasang dengan pip install, lalu MLflow diarahkan ke database SQLite lokal (sqlite:///mlflow.db) sebagai pengganti tracking server.

```
!pip install -q "mlflow>=3" scikit-learn pandas matplotlib requests
```

```
import os, warnings
warnings.filterwarnings("ignore")
import mlflow, mlflow.sklearn

os.makedirs("data", exist_ok=True)
mlflow.set_tracking_uri("sqlite:///mlflow.db")   # pengganti "mlflow server" + export di Colab
print("MLflow", mlflow.__version__)
```

```
MLflow 3.17.0
```

Output menunjukkan MLflow 3.17.0 sudah terpasang. os.makedirs("data") membuat folder untuk menyimpan hasil split, dan mlflow.set_tracking_uri membuat semua run tercatat di satu database yang sama.

3. L1: Data, Split Tetap dan Versi Data (Hash)

```
import hashlib, pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

mlflow.set_experiment("bc-malignant-detection")

df = load_breast_cancer(as_frame=True).frame
df["target"] = 1 - df["target"]                    # 1 = malignant (kelas positif)

# Sidik jari data: berubah satu nilai pun → hash berbeda
data_md5 = hashlib.md5(pd.util.hash_pandas_object(df, index=True).values).hexdigest()
```

```
train_df, test_df = train_test_split(df, test_size=0.2, stratify=df["target"], random_state=42)
train_df.to_csv("data/train.csv", index=False)
test_df.to_csv("data/test.csv", index=False)          # test set: DIKUNCI sampai L6
```

```
X_train, y_train = train_df.drop(columns="target"), train_df["target"]
X_test,  y_test  = test_df.drop(columns="target"),  test_df["target"]

# Objek Dataset MLflow → tampil di tab "Datasets" pada setiap run
train_ds = mlflow.data.from_pandas(train_df, source="data/train.csv",
                                   name="bc-train", targets="target")

COMMON_TAGS = {"data_md5": data_md5, "split_seed": "42", "dataset": "sklearn-breast-cancer"}
```

Kode ini melakukan empat hal. Pertama, label dibalik (1 - target) agar malignant menjadi kelas positif. Kedua, data_md5 dihitung dari seluruh dataframe sebagai sidik jari data: berubah satu nilai saja, hash-nya berbeda. Ketiga, data dibagi 80/20 secara stratified dengan random_state=42, dan hasilnya disimpan ke data/train.csv dan data/test.csv; test set dikunci sampai L6. Keempat, objek dataset MLflow bc-train dibuat agar muncul di tab *Datasets* pada setiap run, dan COMMON_TAGS (data_md5, split_seed, dataset) dipasang ke semua run.

4. Fungsi Logging Standar run_experiment()

```
import mlflow, matplotlib.pyplot as plt
from sklearn.model_selection import StratifiedKFold, cross_validate, cross_val_predict
from sklearn.metrics import ConfusionMatrixDisplay, classification_report
from mlflow.models import infer_signature

CV = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
SCORING = ["recall", "precision", "f1", "roc_auc"]

# fold identik utk semua run
def run_experiment(run_name, pipe, tags=None, note=""):
  with mlflow.start_run(run_name=run_name) as run:
    # (a) KONTEKS: tag, catatan hipotesis, versi data
    mlflow.set_tags({**COMMON_TAGS, "candidate": "true", **(tags or {})})
    mlflow.set_tag("mlflow.note.content", note)        # deskripsi run di UI
    mlflow.log_input(train_ds, context="training")
    # (b) PARAMETER: langkah pipeline + seluruh hyperparameter estimator
    mlflow.log_param("pipeline_steps", " -> ".join(pipe.named_steps))
    mlflow.log_params({f"clf__{k}": v for k, v in pipe[-1].get_params().items()})

    # (c) METRIK: mean & std dari 5-fold CV (bukan satu angka tunggal!)
    cv = cross_validate(pipe, X_train, y_train, cv=CV, scoring=SCORING)
    for m in SCORING:
      mlflow.log_metric(f"cv_{m}_mean", cv[f"test_{m}"].mean())
      mlflow.log_metric(f"cv_{m}_std",  cv[f"test_{m}"].std())
```

```
    # (d) ARTEFAK DIAGNOSTIK: prediksi out-of-fold (tanpa menyentuh test set)
    y_oof = cross_val_predict(pipe, X_train, y_train, cv=CV)
    disp = ConfusionMatrixDisplay.from_predictions(
        y_train, y_oof, display_labels=["benign", "malignant"])
    mlflow.log_figure(disp.figure_, "plots/confusion_matrix_oof.png")
    plt.close(disp.figure_)
    mlflow.log_text(classification_report(y_train, y_oof, zero_division=0),
                    "reports/classification_report_oof.txt")

    # (e) MODEL: dilatih ulang di seluruh train set + signature & contoh input
    pipe.fit(X_train, y_train)
    signature = infer_signature(X_train, pipe.predict(X_train))
    info = mlflow.sklearn.log_model(pipe, name="model", signature=signature,
                                    input_example=X_train.head(3),
                                    skops_trusted_types=["sklearn.tree._tree.Tree"])  # utk DT/RF/GB
    mlflow.set_tag("model_uri", info.model_uri)        # models:/m-… (MLflow 3)
    return run.info.run_id
```

Satu fungsi dipakai oleh semua eksperimen, sehingga nama metrik dan artefak konsisten dan mudah dibandingkan. Fold CV juga identik untuk semua run (StratifiedKFold, 5 fold, random_state=42). Yang dicatat per run:

Yang tercatat	**Lokasi di MLflow**
data_md5, stage, model_family, catatan hipotesis	Tags / Description
Git commit, nama script, user	Otomatis: mlflow.source.git.commit, mlflow.source.name, mlflow.user
Dataset bc-train	Tab *Datasets used*
Hyperparameter dan langkah pipeline	Parameters
cv_*_mean, cv_*_std, fit_time_s	Metrics
Confusion matrix, classification report, model + requirements.txt + conda.yaml	Artifacts

Fungsi bekerja dalam lima bagian.

(a) *Konteks*: tag, catatan hipotesis, dan input dataset dicatat.
(b) *Parameter*: langkah pipeline dan seluruh hyperparameter estimator.
(c) *Metrik*: mean dan standar deviasi dari 5-fold CV untuk recall, precision, F1, dan ROC-AUC, bukan satu angka tunggal.
(d) *Artefak diagnostik*: confusion matrix dan classification report dari prediksi out-of-fold, tanpa menyentuh test set.

(e) *Model*: dilatih ulang pada seluruh train set, disimpan bersama signature dan contoh input.

5. L2: Baseline, Seberapa Buruk "Tanpa Model"? (H1, H2)

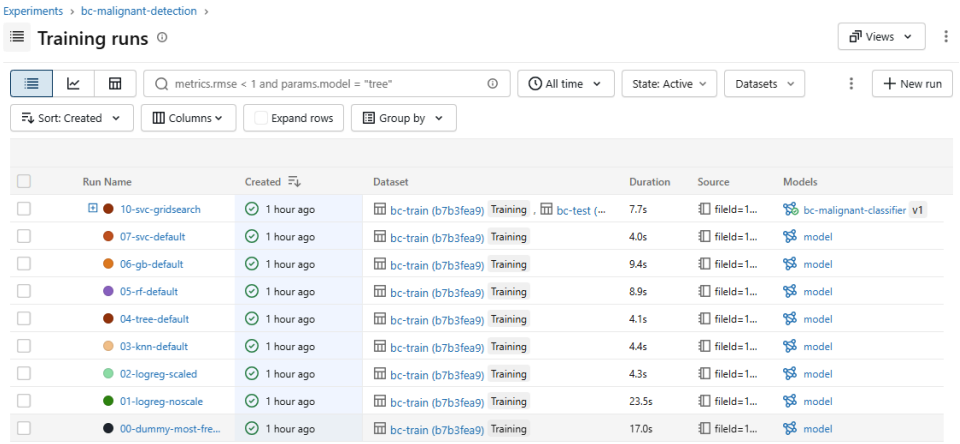
```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression

run_experiment("00-dummy-most-frequent",
    Pipeline([("clf", DummyClassifier(strategy="most_frequent"))]),
    tags={"stage": "baseline", "model_family": "dummy"},
    note="Batas bawah: selalu menebak kelas mayoritas (benign).")

run_experiment("01-logreg-noscale",
    Pipeline([("clf", LogisticRegression(max_iter=5000))]),
    tags={"stage": "baseline", "model_family": "logreg"},
    note="H1: model linear tanpa preprocessing.")

run_experiment("02-logreg-scaled",
    Pipeline([("scaler", StandardScaler()), ("clf", LogisticRegression(max_iter=5000))]),
    tags={"stage": "baseline", "model_family": "logreg"},
    note="H2: hanya menambah StandardScaler, variabel lain tetap.")
```

```
'9d81d9a8068e4743bef1bd7340aef905'
```



Metric	Value
cv_recall_mean	0.9529411764705882
cv_precision_mean	0.9771428571428572

Tiga run baseline dibuat untuk menguji H1 dan H2: 00-dummy-most-frequent (selalu menebak kelas mayoritas, yaitu benign), 01-logreg-noscale (Logistic Regression tanpa preprocessing), dan 02-logreg-scaled (sama persis dengan 01 ditambah StandardScaler).

Output notebook hanya menampilkan ID run terakhir ("9d81d9a8068e4743bef1bd7340aef905"), sedangkan hasilnya dibaca di MLflow UI. Halaman Training runs menampilkan seluruh run beserta dataset bc-train, dan run 02-logreg-scaled mencatat cv_recall_mean 0,953 serta cv_precision_mean 0,977.

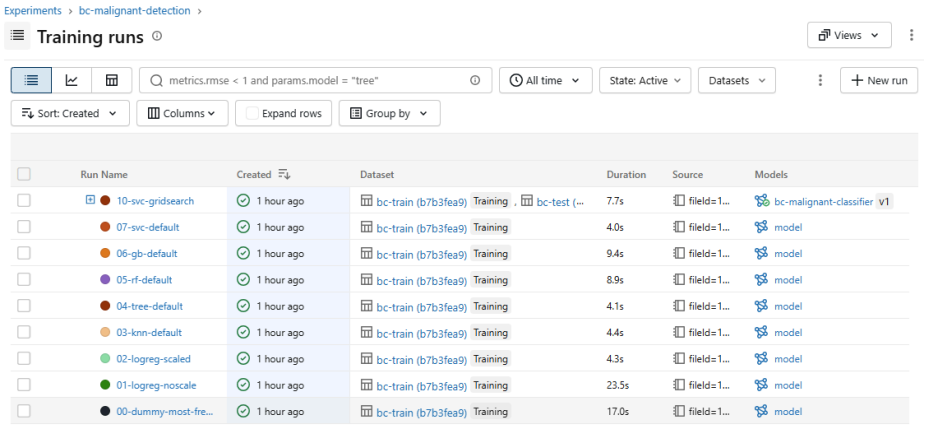
- **Dummy:** model ini tidak mendeteksi satu pun tumor ganas, sehingga recall-nya 0. Namun akurasinya tetap terlihat tinggi karena kelas benign mayoritas. Ini bukti bahwa accuracy menyesatkan, dan alasan recall dipilih sebagai optimizing metric.
- **H1:** Logistic Regression sederhana jauh melampaui dummy, dari recall 0 menjadi 0,918 tanpa scaling. H1 didukung.
- **H2:** Run 01 dan 02 hanya berbeda di scaling. Recall naik dari 0,918 menjadi 0,953, jadi kenaikannya berasal dari standardisasi. H2 didukung (diuji langsung pada *Logistic Regression*).

6. L3: Model Selection, Bandingkan Algoritma Sederhana

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
```

```
candidates = {
    "knn":  KNeighborsClassifier(n_neighbors=5),
    "tree": DecisionTreeClassifier(max_depth=5, random_state=42),
    "rf":   RandomForestClassifier(n_estimators=200, random_state=42),
    "gb":   GradientBoostingClassifier(random_state=42),
    "svc":  SVC(kernel="rbf", random_state=42),
}

for i, (family, clf) in enumerate(candidates.items(), start=3):
    pipe = Pipeline([("scaler", StandardScaler()), ("clf", clf)])
    run_experiment(f"{i:02d}-{family}-default", pipe,
                   tags={"stage": "model-selection", "model_family": family},
                   note="Hyperparameter default; preprocessing identik untuk semua.")
```



Lima algoritma dibandingkan: KNN (n_neighbors=5), Decision Tree (max_depth=5), Random Forest (200 pohon), Gradient Boosting, dan SVC (kernel RBF), semuanya dengan hyperparameter sederhana tanpa tuning dan StandardScaler yang sama. Nomor run berurutan dari 03 sampai 07.

Perbandingan ini adil karena fold CV sama, preprocessing sama, dan data sama (data_md5 sama), sehingga perbedaan metrik hanya berasal dari algoritma. Sel ini tidak mencetak output karena hasilnya dicatat otomatis ke MLflow, dan daftar run bisa dilihat di MLflow UI. Pada hasil L5, SVC default dan Gradient Boosting sama-sama mencapai recall CV 0,947, sedikit di bawah Logistic Regression berskala (0,953), sedangkan Random Forest 0,935. Decision Tree tersaring otomatis oleh syarat precision ≥ 0,90 di L5. Nilai metrik masing-masing model ditampilkan pada output L5.

7. L4: Hyperparameter Tuning dengan Nested Runs (H3)

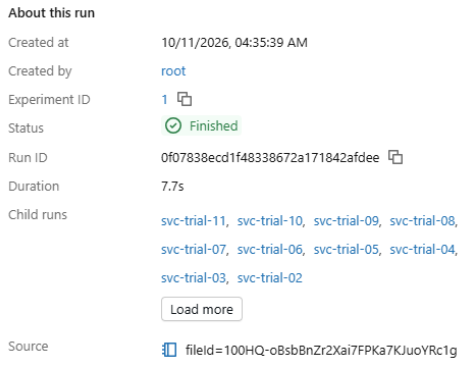
```
from sklearn.model_selection import GridSearchCV

pipe = Pipeline([("scaler", StandardScaler()), ("clf", SVC(kernel="rbf", random_state=42))])
grid = {"clf__C": [0.1, 1, 10, 100], "clf__gamma": ["scale", 0.01, 0.001]}   # 12 kombinasi
gs = GridSearchCV(pipe, grid, cv=CV, scoring=SCORING, refit="recall", n_jobs=-1)
```

```
with mlflow.start_run(run_name="10-svc-gridsearch",
                      tags={**COMMON_TAGS, "stage": "tuning", "model_family": "svc",
                            "candidate": "true"}):
    mlflow.log_input(train_ds, context="training")
    mlflow.log_dict(grid, "search_space.json")                # ruang pencarian = artefak
    gs.fit(X_train, y_train)
    res = gs.cv_results_

    for i, params in enumerate(res["params"]):                # 1 trial = 1 child run
        with mlflow.start_run(run_name=f"svc-trial-{i:02d}", nested=True):
            mlflow.log_params(params)
            mlflow.log_metrics({f"cv_{m}_mean": res[f"mean_test_{m}"][i] for m in SCORING})
            mlflow.log_metrics({f"cv_{m}_std":  res[f"std_test_{m}"][i]  for m in SCORING})
    b = gs.best_index_
```

```
# parent = ringkasan terbaik
mlflow.log_params(gs.best_params_)
mlflow.log_metrics({f"cv_{m}_mean": res[f"mean_test_{m}"][b] for m in SCORING})
mlflow.log_metrics({f"cv_{m}_std":  res[f"std_test_{m}"][b]  for m in SCORING})
info = mlflow.sklearn.log_model(gs.best_estimator_, name="model",
    signature=infer_signature(X_train, gs.best_estimator_.predict(X_train)),
    input_example=X_train.head(3))
mlflow.set_tag("model_uri", info.model_uri)
```



Pada tahap ini SVC di-tuning dengan mencoba semua kombinasi pengaturan C (0,1, 1, 10, 100) dan gamma (scale, 0,01, 0,001), sehingga ada 12 percobaan. Setiap percobaan dievaluasi dengan 5-fold CV yang sama. Run induk 10-svc-gridsearch menyimpan daftar pengaturan yang dicoba (search_space.json), dan tiap percobaan dicatat sebagai *child run* (svc-trial-00 sampai svc-trial-11).

Kombinasi terbaik adalah C=10 dan gamma=scale. Recall CV naik dari 0,947 (SVC default) menjadi 0,965, dan precision 0,971 masih di atas syarat 0,90. Namun, kenaikan 0,018 itu masih lebih kecil dari simpangan baku recall (0,029), jadi belum cukup untuk memastikan tuning benar-benar lebih baik. H3 didukung, tetapi perlu pengujian tambahan seperti repeated CV.

Sebagai alternatif yang lebih singkat, mlflow.sklearn.autolog() bisa mencatat child run GridSearchCV secara otomatis (default 5 terbaik, diatur lewat max_tuning_runs).

8. L5: Analisis Programatik, Pilih Kandidat Terbaik

```
import mlflow

runs = mlflow.search_runs(
    experiment_names=["bc-malignant-detection"],
    filter_string=(
        "tags.candidate = 'true' "                    # hanya run yang punya model
        "and metrics.cv_precision_mean >= 0.90"       # satisficing metric
    ),
    order_by=["metrics.cv_recall_mean DESC",          # optimizing metric
              "metrics.cv_f1_mean DESC"],             # tie-breaker
)

cols = ["tags.mlflow.runName", "tags.model_family",
        "metrics.cv_recall_mean", "metrics.cv_recall_std", "metrics.cv_precision_mean"]
print(runs[cols].head(5).to_string(index=False))

best_run_id    = runs.iloc[0]["run_id"]
best_model_uri = runs.iloc[0]["tags.model_uri"]       # models:/m-…
print("Kandidat terpilih:", runs.iloc[0]["tags.mlflow.runName"], best_run_id)
```

```
tags.mlflow.runName tags.model_family  metrics.cv_recall_mean  metrics.cv_recall_std  metrics.cv_precision_mean
     10-svc-gridsearch               svc                0.964706               0.028818                   0.971092
      02-logreg-scaled            logreg                0.952941               0.039896                   0.977143
         06-gb-default                gb                0.947059               0.034300                   0.964631
        07-svc-default               svc                0.947059               0.034300                   0.976946
         05-rf-default                rf                0.935294               0.039019                   0.963759
Kandidat terpilih: 10-svc-gridsearch 0f07838ecd1f48338672a171842afdee
```

Filter: hanya run bertag candidate = 'true' dengan cv_precision_mean ≥ 0,90 (satisficing metric), diurutkan menurut cv_recall_mean (optimizing metric) lalu cv_f1_mean sebagai pemutus seri. Aturan yang sama dapat dijalankan ulang otomatis di pipeline Continuous Training.

Run	Keluarga	Recall CV (mean)	Recall CV (std)	Precision CV (mean)
10-svc-gridsearch	svc	0,9647	0,0288	0,9711
02-logreg-scaled	logreg	0,9529	0,0399	0,9771
06-gb-default	gb	0,9471	0,0343	0,9646
07-svc-default	svc	0,9471	0,0343	0,9769
05-rf-default	rf	0,9353	0,0390	0,9638

Kandidat terpilih adalah 10-svc-gridsearch, yang juga menjadi best_model_uri untuk tahap berikutnya.

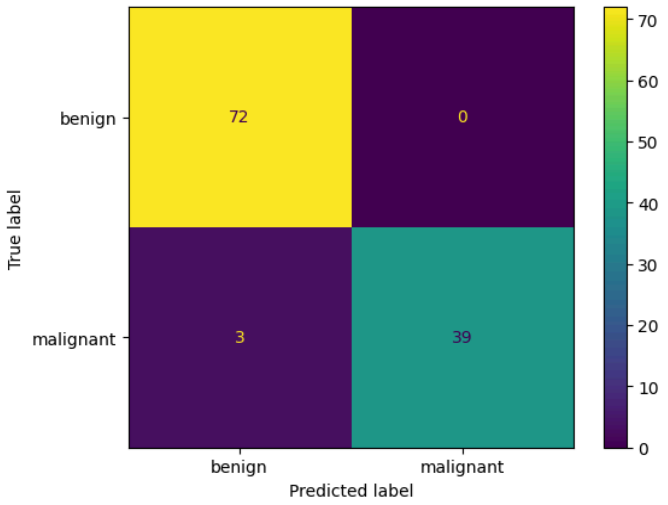
9. L6: Evaluasi Final di Test Set (Hanya Sekali)

```
from sklearn.metrics import recall_score, precision_score, f1_score, roc_auc_score
from sklearn.metrics import ConfusionMatrixDisplay

model = mlflow.sklearn.load_model(best_model_uri)
y_pred = model.predict(X_test)
score = (model.predict_proba(X_test)[:, 1] if hasattr(model, "predict_proba")
         else model.decision_function(X_test))
```

```
test_metrics = {
    "test_recall":    recall_score(y_test, y_pred),
    "test_precision": precision_score(y_test, y_pred),
    "test_f1":        f1_score(y_test, y_pred),
    "test_roc_auc":   roc_auc_score(y_test, score),
}

with mlflow.start_run(run_id=best_run_id):              # tambahkan ke run yang SAMA
    mlflow.log_metrics(test_metrics)
    mlflow.log_input(mlflow.data.from_pandas(test_df, source="data/test.csv",
                     name="bc-test", targets="target"), context="testing")
    disp = ConfusionMatrixDisplay.from_predictions(y_test, y_pred,
                     display_labels=["benign", "malignant"])
    mlflow.log_figure(disp.figure_, "plots/confusion_matrix_test.png")
    mlflow.set_tag("test_evaluated", "true")
```



Model dari best_model_uri diuji satu kali pada test set yang tadinya dikunci. Hasilnya ditambahkan ke run yang sama, bersama dataset bc-test, confusion matrix, dan tag test_evaluated = true. Jadi satu run berisi hasil CV, hasil test, data, dan model.

Metrik test	**Nilai**
Recall	0,929
Precision	1,000
F1	0,963
ROC-AUC	0,993

Precision test 1,000: semua tumor yang diprediksi ganas memang ganas, tidak ada alarm palsu (tumor jinak yang salah diprediksi ganas). Recall test 0,929: dari 42 tumor ganas, 39 terdeteksi dan 3 terlewat. Recall test sedikit di bawah recall CV 0,965 (selisih 0,036), tapi tidak jauh, jadi model tidak tampak overfit pada seleksi. Kalau selisihnya besar, model tidak boleh di-tuning ulang memakai test set.

10. L7: Model Registry dan Quality Gate Otomatis

```
from mlflow import MlflowClient
client, NAME = MlflowClient(), "bc-malignant-classifier"

mv = mlflow.register_model(best_model_uri, NAME)                    # → versi baru
client.update_model_version(NAME, mv.version,
    description=f"Dipilih otomatis: max cv_recall dgn precision>=0.90. data_md5={data_md5}")
client.set_model_version_tag(NAME, mv.version, "data_md5", data_md5)
client.set_registered_model_alias(NAME, "challenger", mv.version)

def champion_recall():
  try:
      champ = client.get_model_version_by_alias(NAME, "champion")
      return client.get_run(champ.run_id).data.metrics["test_recall"]
  except Exception:                                    # belum ada champion
      return -1.0

MIN_PRECISION, MARGIN = 0.90, 0.005
passed = (test_metrics["test_precision"] >= MIN_PRECISION and
          test_metrics["test_recall"] >= champion_recall() + MARGIN)
```

```
MIN_PRECISION, MARGIN = 0.90, 0.005
passed = (test_metrics["test_precision"] >= MIN_PRECISION and
          test_metrics["test_recall"] >= champion_recall() + MARGIN)

client.set_model_version_tag(NAME, mv.version, "validation_status",
                             "approved" if passed else "rejected")
if passed:
    client.set_registered_model_alias(NAME, "champion", mv.version)   # promosi
```

```
Successfully registered model 'bc-malignant-classifier'.
Created version '1' of model 'bc-malignant-classifier'.
```

Model didaftarkan di Model Registry dengan nama bc-malignant-classifier dan menjadi versi 1. Versi ini diberi deskripsi, tag data_md5, dan alias challenger (calon model).

Sebelum dipakai, model diperiksa otomatis lewat quality gate dengan dua syarat: test_precision minimal 0,90, dan test_recall harus lebih tinggi minimal 0,005 dari champion saat ini. Karena belum ada champion, nilai pembandingnya −1, jadi syarat kedua otomatis terpenuhi. Dengan precision 1,000 dan recall 0,929, model lolos, tag validation_status menjadi approved, dan versi 1 mendapat alias champion.

Perbandingan ini adil hanya kalau champion dan challenger diuji pada test set yang sama (cek tag data_md5). Kalau model baru bermasalah, alias champion bisa dikembalikan ke versi sebelumnya dengan set_registered_model_alias.

11. L8: Serving Model dari Registry

```
import subprocess, time
env = {**os.environ, "MLFLOW_TRACKING_URI": "sqlite:///mlflow.db"}
srv = subprocess.Popen(["mlflow", "models", "serve", "-m", "models:/bc-malignant-classifier@champion",
                        "-p", "5001", "--env-manager", "local"],
                       env=env, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)
time.sleep(30)   # tunggu server siap
```

```
import requests

payload = {"dataframe_split": X_test.head(3).to_dict(orient="split")}
r = requests.post("http://127.0.0.1:5001/invocations", json=payload)
print(r.json())                    # {"predictions": [0, 1, 0]}  (1 = malignant)

# Atau langsung di aplikasi Python (batch scoring)
model = mlflow.pyfunc.load_model("models:/bc-malignant-classifier@champion")
preds = model.predict(X_test)
# input divalidasi terhadap signature

srv.terminate()   # matikan server setelah selesai
```

```
{'predictions': [0, 1, 0]}
```

Pada tahap ini model dijalankan sebagai layanan (*serving*). Model dimuat lewat alias, yaitu models:/bc-malignant-classifier@champion, bukan lewat path file atau nomor versi. Di komputer lokal, server dijalankan dengan mlflow models serve; di Colab, server dijalankan di background lewat subprocess.

Untuk mengujinya, tiga baris pertama test set dikirim ke endpoint /invocations, dan server membalas {'predictions': [0, 1, 0]} (1 = malignant, 0 = benign). Model juga bisa dimuat langsung dengan mlflow.pyfunc.load_model untuk memprediksi banyak data sekaligus (batch scoring).

Model menyimpan signature, yaitu daftar kolom dan tipe data yang diharapkan, sehingga input yang salah ditolak dan tidak menghasilkan prediksi keliru secara diam-diam. Kalau champion berganti (L7), layanan cukup di-restart tanpa mengubah kode.

12. L9: Reproduce, dari Model di Produksi Kembali ke Asal-usulnya

```
champ = client.get_model_version_by_alias("bc-malignant-classifier", "champion")
run = client.get_run(champ.run_id)

print("Run        :", run.info.run_name, run.info.run_id)
print("Git commit :", run.data.tags.get("mlflow.source.git.commit"))
print("Data md5   :", run.data.tags["data_md5"])
print("Params     :", {k: v for k, v in run.data.params.items() if k.startswith("clf__")})
print("Metrics    :", {k: round(v, 3) for k, v in run.data.metrics.items()})

# 1) Pastikan data saat ini identik dengan data training
assert data_md5 == run.data.tags["data_md5"], "Data berbeda → hasil tidak akan sama!"

# 2) Unduh artefak (model, requirements.txt, plot) untuk audit
path = mlflow.artifacts.download_artifacts(run_id=champ.run_id, artifact_path="model")
print(open(f"{path}/requirements.txt").read())
```

```
Run        : 10-svc-gridsearch 0f07838ecd1f48338672a171842afdee
Git commit : None
Data md5   : ee5481ad12c0896439e993aacc9e8d50
Params     : {'clf__C': '10', 'clf__gamma': 'scale'}
Metrics    : {'cv_recall_mean': 0.965, 'cv_precision_mean': 0.971, 'cv_f1_mean': 0.967, 'cv_roc_auc_mean': 0.993, 'cv_recall_std': 0.029, 'cv_precision
Downloading artifacts:   0% 0/1 [00:00<?, ?it/s]
Downloading artifacts: 100% 8/8 [00:00<00:00, 328.58it/s]
mlflow==3.17.0
lz4==4.4.5
numpy==2.1.3
pandas==2.2.3
polars==1.35.2
psutil==5.9.5
pytest==8.4.2
scikit-learn==1.6.1
scipy==1.16.3
skops==0.16.0
```

Dari alias champion, kita bisa menelusuri model produksi kembali ke asalnya: model v1 (@champion) berasal dari run 10-svc-gridsearch, yang menyimpan git commit, data_md5, hyperparameter, dan requirements.txt. Hasil output:

- **Run:** 10-svc-gridsearch (0f07838ecd1f48338672a171842afdee).
- **Data md5:** ee5481ad12c0896439e993aacc9e8d50. Lewat assert, kode memastikan data sekarang sama dengan data training.
- **Parameter:** C=10, gamma=scale.
- **Metrik:** recall CV 0,965, precision CV 0,971, recall test 0,929, precision test 1,0, ROC-AUC test 0,993.
- **Environment:** mlflow==3.17.0, scikit-learn==1.6.1, numpy==2.1.3, pandas==2.2.3, dan library lainnya.
- **Git commit:** None, karena eksperimen dijalankan di Colab yang tidak memakai repositori git. Kalau dijalankan di repositori lokal, commit tercatat otomatis. Dengan commit itu, hasil yang sama bisa diulang lewat git checkout <commit>, data dengan hash yang sama, environment yang sama, dan random_state yang sama.

Dengan begitu, setiap prediksi di produksi bisa dilacak sampai ke data, parameter, dan library yang dipakai.

13. Kesimpulan Hipotesis dan Pemetaan ke Prinsip MLOps

Hipotesis	Hasil	Status
H1: model linear jauh lebih baik dari dummy	Recall 0,000 (dummy) menjadi 0,918 (LogReg)	Didukung
H2: standardisasi meningkatkan recall	LogReg 0,918 menjadi 0,953; SVC tanpa tuning 0,947	Didukung
H3: tuning SVC meningkatkan recall tanpa melanggar precision ≥ 0,90	Recall CV 0,947 menjadi 0,965, precision 0,971	Didukung secara arah; selisih masih dalam std 0,029

Model akhir (SVC, C=10, gamma=scale) lolos quality gate: recall 0,929, precision 1,000, ROC-AUC 0,993.

Langkah	Praktik yang Diterapkan	Prinsip MLOps
Skenario	Hipotesis dan metrik ditetapkan di awal	Design, Iterative development
L1	Split tetap, data_md5, log_input	Versioning, Reproducibility
L2–L3	Baseline, controlled experiment, CV mean ± std	Experiments Tracking
L4	GridSearchCV + nested runs, search space sebagai artefak	Automation, Experiments Tracking
L5	Seleksi model via search_runs (aturan tertulis)	Automation, Continuous Training
L6	Test set disentuh sekali, metrik test ditambahkan ke run yang sama	Testing
L7	Registry, alias, quality gate, rollback	Versioning, Continuous Delivery
L8	Serving via models:/…@champion + signature	Deployment, Loosely coupled
L9	Lineage dari prediksi → run → commit dan data	Reproducibility, Governance